HelixCode — Open Issues Tracker

> Per Constitution §11.4.15 (Item-status tracking) + §11.4.16 (Item-type tracking) + §11.4.19 (Fixed-document column-alignment) + CONST-057 (Type-aware closure vocabulary) + CONST-058 (Reopened-source attribution).
>
> **Authoritative resumption ledger**: `docs/CONTINUATION.md` (CONST-044). This file complements it with item-level granularity for currently-open work.
>
> **Status vocabulary** (closed set): `Queued` | `In progress` | `Ready for testing` | `In testing` | `Reopened` | `Fixed/Implemented/Completed (→ Fixed.md)`
>
> **Type vocabulary** (closed set): `Bug` | `Feature` | `Task`

Prefix convention

Round 189 (2026-05-19) introduced per-project / per-submodule ID prefixes replacing the legacy `ISSUE-NNN` flat namespace. New items MUST use the prefix matching their scope; cross-cutting items affecting two or more submodules (or any meta-repo concern) live under `HXC`. Numeric portion is per-prefix (each prefix starts at `001`). Legacy `ISSUE-NNN` IDs renamed forward-only per CONST-043; historical close-out narratives in `docs/CONTINUATION.md` preserve original IDs verbatim, and `docs/Fixed.md` annotates each closure with `(ex-ISSUE-NNN)` for git-history traceability.

Prefix	Scope	Source
HXC	HelixCode root project (project-wide, multi-submodule,	this repo

Prefix	Scope	Source
	governance, infrastructure)	
HXA	HelixAgent submodule	`submodules/helix_agent` (when present) / `helix_agent/` tree
HXM	HelixMemory submodule	`submodules/helix_memory`
HXL	HelixLLM submodule	`submodules/helix_llm`
HXQ	HelixQA submodule	`submodules/helix_qa` (`helix_qa/`)
HXS	HelixSpecifier submodule	`submodules/helix_specifier`
HXO	HelixOrchestrator (= LLMOrchestrator) submodule	`submodules/llm_orchestrator`
HXV	HelixVerifier (= LLMsVerifier) submodule	`submodules/llms_verifier`
HXD	HelixDocProcessor (= DocProcessor) submodule	`submodules/doc_processor`
HXI	HelixI18n (when added)	tba
PLN	Planning submodule (vasic-digital)	`submodules/planning`
VEN	VisionEngine submodule (HelixDevelopment)	`submodules/vision_engine`
SLF	SelfImprove submodule (vasic-digital)	`submodules/self_improve`
STO	Storage submodule (vasic-digital)	`submodules/storage`
OPS	LLMOps submodule (vasic-digital)	`submodules/llm_ops`

Prefix	Scope	Source
VDB	VectorDB submodule (vasic-digital)	`submodules/vector_db`
OBS	Observability submodule (vasic-digital)	`submodules/observability`
MCP	MCP_Module submodule (vasic-digital)	`submodules/mcp_module`
MSG	Messaging submodule (vasic-digital)	`submodules/messaging`
MDW	Middleware submodule (vasic-digital)	`submodules/middleware`
PLG	Plugins submodule (vasic-digital)	`submodules/plugins`
STR	Streaming submodule (vasic-digital)	`submodules/streaming`
WAT	Watcher submodule (vasic-digital)	`submodules/watcher`
CNV	conversation submodule (vasic-digital)	`submodules/conversation`
AUT	Auth submodule (vasic-digital)	`submodules/auth`
LZY	Lazy submodule (vasic-digital)	`submodules/lazy`
ATP	AutoTemp submodule (vasic-digital)	`submodules/auto_temp`
PLI	PliniusCommon submodule (vasic-digital)	`submodules/plinius_common`
CHL	challenges submodule (vasic-digital)	`challenges/`
CNT		`containers/`

Prefix	Scope	Source
	containers submodule	
SEC	security submodule	`security/`
PAN	panoptic submodule	`panoptic/`

For submodules not listed above, default to the first 3 letters of the submodule name, uppercase (e.g. `Watcher` → `WAT`). Document the new prefix in this table on first use.

Legacy → new mapping (round 189)

Old ID	New ID	Scope rationale
ISSUE-001	VEN-001	VisionEngine `helix-gitlab` URL
ISSUE-002	VEN-002	VisionEngine `vasic-digital-github` fork divergent
ISSUE-003	HXL-001	HelixLLM `analysis_test.go` hardcoded path
ISSUE-004	HXL-002	HelixLLM TOON `WriteTOON` 500
ISSUE-005	HXC-001	CONST-052 rename programme (project-wide)
ISSUE-006	HXC-002	Round-74 residual LOGIC FAILs (multi-submodule cross-cutting)
ISSUE-007	HXC-003	CONST-046 migration backlog (project-wide)
ISSUE-008	HXQ-001	helix_qa `TestPerformance` flake
ISSUE-009	HXA-001	helix_agent handler tests (4)
ISSUE-010	HXA-002	helix_agent debate API drift
ISSUE-011	HXA-003	venice CONST-037 (in helix_agent)

HXC-159 — Fully incorporate the HelixSkills System into HelixCode + HelixAgent (all power-features, SpecKit-driven)

Status: Queued **Type:** Task **Severity:** High **Created-By:** Operator

Reported-Via: §11.4.202 reporting directive `task` on 2026-07-28T19:26:22Z **Reported-By:** Operator

What (the report, verbatim): Feature description (verbatim): Fully incorporate the HelixSkills System (git@github.com:HelixDevelopment/skills.git) into BOTH HelixCode and HelixAgent, covering ALL of its power-features with nothing left out.

VERBATIM OPERATOR REQUIREMENTS (2026-07-29): “Fully incorporate into HelixCode and HelixAgent the HelixSkills System for all work with Skills: git@github.com:HelixDevelopment/skills.git , we MUST fully incorporate all HelixSkills power-features fully without nothing leftout! Full exhaustive implementation plan divided into phases, tasks and subtasks with exhaustive amount of data with all details MUST BE prepared! Full coverage with ll supported test types, challenges and HelixQA test banks (suites) is mandatory! Extend and update all existing documentation, user manuals, guides, FAQs, create stunning illustrations, graphs and diagrams and incorporate them in all materials! Any gaps, shortcomings, weak spots, danger zones, issues, or any inconsistencies MUST BE detected in advance during the planning and implementation process and properly tackled with comprehensive rock-solid solutions implementations, risk-free and safe! Track all progress through the constinuation docs, memory, knowledge and remebering mechanisms we have! Use GitHub SpecKit for all phases of incorporating with bridge to Superpowers used for the implementation and subagents driven development mechanism!”

PRE-VERIFIED ENVIRONMENT FACTS (established 2026-07-29, not assumed — §11.4.6): - GitHub SpecKit IS installed at /home/milos/.local/bin/specify. It is NOT yet initialized in this repo (no .specify/ and no specs/ directory present). Initialization is therefore an explicit early task, not an assumption. - git@github.com:HelixDevelopment/skills.git IS reachable via SSH. `git ls-remote --heads` returns branches including feature/catalog-docs, feature/deep-research, feature/testing-infra. The default branch and full branch/tag inventory MUST be re-derived at research time rather than inferred from this partial listing. - The repository is NOT currently a submodule of this project. `.gitmodules` contains no HelixDevelopment/skills entry; the only ‘skills’-matching entry is cli_agents/codex-skills -> vasic-digital/caf-codex-skills.git, which is a DIFFERENT repository and must not be confused with it. - Both consumers exist in-tree: the inner Go module helix_code/ (module dev.helix.code) and the submodule submodules/helix_agent (module dev.helix.agent).

MANDATORY DELIVERY CONSTRAINTS: - SpecKit-first: use GitHub SpecKit for ALL phases of the incorporation, bridged to Superpowers for implementation, executed subagent-driven (§11.4.70 / §11.4.20). - Exhaustive plan structured as phases -> tasks -> subtasks, with an exhaustive level of detail (down to lines-of-code and micro-POCs where the design is load-bearing). - Complete test coverage per §11.4.169: unit, integration, e2e, full-automation, Challenges (vasic-digital/challenges), HelixQA banks (HelixDevelopment/helix_qa) with autonomous sessions, DDoS, security, stress + chaos, concurrency/atomicity, race/deadlock, memory, benchmarking. Every PASS carries captured evidence (§11.4.5 / §11.4.69); every guard ships a paired §1.1 mutation. - Documentation: extend AND update every existing doc, user manual, guide and FAQ — not only new ones. Produce illustrations, graphs and diagrams and incorporate them into all materials. Four-format export discipline per §11.4.65 / §11.4.153. Every doc reachable from the main README per §11.4.212. - Risk work is a PLANNING deliverable, not a post-hoc note: every gap, shortcoming, weak spot, danger zone, issue and inconsistency MUST be detected DURING planning and each one paired with a comprehensive, rock-solid, risk-free and safe solution (§11.4.102 systematic-debugging over all gathered data). - Reuse before rewrite (§11.4.74 / §11.4.28): survey the vasic-digital and HelixDevelopment catalogues on GitHub AND GitLab first; extend owned submodules in place rather than duplicating, keeping them project-agnostic and decoupled. HelixSkills itself must be incorporated as a properly-laid-out dependency per §11.4.28(C) / CONST-051(C) — root-level or submodules/, never a nested own-org chain. - Cross-consumer parity: HelixCode and HelixAgent must BOTH receive the full feature set; a capability landing in one and not the other is an incompleteness to be tracked, not silently accepted. - Progress tracked through the continuation docs, the memory/knowledge/remembering mechanisms, and the workable-items SSoT (§12.10 / §11.4.131 / §11.4.93 / §11.4.95).

ACCEPTANCE CRITERIA: 1. A SpecKit specification exists and is initialized in-repo, covering every HelixSkills power-feature, with an explicit feature inventory proven complete against the upstream repository (an enumerated list, not a sample — §11.4.118). 2. A phase/task/subtask implementation plan exists at the stated depth, with each risk item paired to a concrete mitigation. 3. Every planned capability is wired and end-user-reachable in BOTH consumers, proven by §11.4.108 runtime signatures on a clean target — not merely compiled. 4. The full §11.4.169 test-type matrix is green with captured evidence, Challenges and HelixQA banks included. 5. All documentation is updated and exported, with the produced diagrams/illustrations

incorporated, and reachable from README. 6. Independent code review reaches a zero-finding GO per §11.4.125 / §11.4.134 / §11.4.142. 7. No capability is claimed without runtime evidence — the anti-bluff covenant governs every closure (§11.4 / §11.4.1 / §11.4.123).

§11.4.213 FEATURE research-and-planning WORK PROGRAM (scheduled, not yet executed):

1. Deep web-research series (articles / guides / scientific papers / open-source projects+codebases / real-world examples) on the best way to incorporate this feature into the project (§11.4.8 / §11.4.99 / §11.4.150).
2. Systematic-debug (§11.4.102) of ALL data obtained to enumerate EVERY weak spot / gap / danger-zone / inconsistency / imperfection, and design a risk-free, rock-solid solution for EACH one found.
3. The design MUST ALWAYS be enterprise-grade, bleeding-edge, and innovative – never less.
4. MANDATORY exhaustive documentation: many technical documents, an implementation plan down to lines-of-code + micro-proof-of-concepts, diagrams / schemes / graphs, illustrations, SQL definitions, templates, and every other relevant material (§11.4.65 / §11.4.73).
5. Investigate + plan REUSE of existing vasic-digital + HelixDevelopment submodules/components BEFORE proposing a rewrite; extend them freely where genuinely needed while keeping them fully decoupled / project-not-aware / reusable (§11.4.28 / §11.4.74 / §11.4.177).
6. Plan from the TESTING point of view from day one: every supported test type, the Challenges submodule, and full HelixQA bank coverage (§11.4.27 / §11.4.169).
7. Divide the work into phases / tasks / subtasks, fine-grained and nano-detailed enough to drive integration / implementation / wiring / testing / scaling directly.
8. Plan explicitly for enterprise scalability and maximal performance.
9. When the feature has a UI/UX surface, produce full wireframes / diagrams / design files (Figma / PSD / PDF, or whatever the project mandates) authored via OpenDesign (§11.4.162 / §11.4.190).
10. Plan full CodeGraph integration with regular / real-time index synchronisation (§11.4.78 / §11.4.79 / §11.4.80).
11. Create fully-detailed follow-on workable items (every detail + reference + attachment) synced in REAL TIME to the SQLite single-source-of-truth (§11.4.93 / §11.4.95) + every derived workable-items document + every related project doc/component + every connected external work-tracking system (§11.4.148 D5) – honestly SKIPPING any absent tracker with a machine-readable reason

(credentials_absent / tracker_client_absent, §11.4.10) rather than EVER faking a push.

Research-doc destination (to be populated when this item is worked): docs/research/fully_incorporate_the_helixskills_system_into_helixcode_heli_20260728T192622Z_2557683/

Execution model (§11.4.213 clauses 1-2): this item is SCHEDULED, not yet executed. The multi-day research/planning/documentation work above is performed LATER by the project's standing autonomous loop (§11.4.87 / §11.4.94 / §11.4.97 / §11.4.103 / §11.4.126) when it claims this item, and MUST be driven to a genuinely COMPLETED-and-wired or explicitly evidence-backed CLOSED terminal state under §11.4.197 – this item MUST NOT be left sitting un-wired in the backlog.

Affected scope / file-scope manifest: Feature research/planning – destination: docs/research/fully_incorporate_the_helixskills_system_into_helixcode_heli_20260728T192622Z_2557683/

Reproduction / context: UNKNOWN: not stated in the report — a reproduction MUST be established before any fix (§11.4.102 / §11.4.146 / §11.4.199)

Acceptance criteria: The full research-doc tree exists under docs/research/fully_incorporate_the_helixskills_system_into_helixcode_heli_20260728T192622Z_2557683/, every enumerated weak-spot/gap/danger-zone carries a designed risk-free solution, an implementation plan down to lines-of-code exists, the planned follow-on workable items are created, and the whole effort is validated per §11.4.197 (never left un-wired).

=== ADDENDUM — BINDING OPERATOR REQUIREMENT (2026-07-29, added after scheduling) ===

VERBATIM: "Make sure that SpecKit and Superpowers are used with Bridge extension and with all extensons we have created derrived from constitution Submodule!"

This is not optional colour on the SpecKit mandate — it names specific existing assets that this work MUST compose with rather than duplicate (§11.4.74 extend-don't-reimplement).

THE BRIDGE EXTENSION (verified to exist, 2026-07-29): constitution/docs/research/extensions/speckit_superpowers/implementation/ — the "Helix Constitution-Powered SpecKit-Superpowers Bridge". Eight

documents, each in .md/.html/.pdf/.docx: README, IMPLEMENTATION_PLAN, NANO_TASK_ENGINE, EXTENSION_DEVELOPMENT, TDD_INTEGRATION, CONSTITUTION_INTEGRATION, SECURITY, APPENDIX.

It defines a SEVEN-LAYER architecture: 1 Developer Workstation 2 Spec-Kit Core (governance) 3 SuperSpec bridge (orchestration — github.com/WangX0111/superspec) 4 SuperB extension (discipline enforcement — speckit-community.github.io/extensions/superb) 5 SuperBridge MCP (execution) 6 Helix LLM (inference — github.com/HelixDevelopment/helix_llm) 7 Distributed host cluster (llama.cpp RPC)

CONSTITUTION-DERIVED EXTENSIONS THAT MUST BE IN SCOPE (inherited BY REFERENCE per §11.4.228 — never copied locally, a copy diverges silently): constitution/skills/ (7) action-prefix-system, media-validator, multitrack, reporting-workable-items, scheduled-work-queue, session-sync, workable-item-lifecycle constitution/mcp/ (2) media-validator-mcp.json, scheduled-work-mcp.json constitution/plugins/ (2) helix, scheduled-work constitution/actions/ registry.yaml (§11.4.140 action system), subagent_tiering.yaml constitution/scripts/hooks/ (7 live) action_prefix_expand.sh, credential_scan_lib.sh, guard-branch-consistency.sh, guard-forbidden-commands.sh, guard-track-branch-label.sh, guard-work-track-binding.sh, post-merge

ALREADY PRESENT LOCALLY — do not re-create: .claude/skills/ 10 registered speckit-* skills (analyze, checklist, clarify, constitution, converge, implement, plan, specify, tasks, taskstoissues) .specify/workflows/speckit/workflow.yml, integrations/{claude,speckit}.manifest.json, scripts/bash/, templates/, memory/

ADDED ACCEPTANCE CRITERIA: 8. Every constitution-derived extension above has an explicit disposition — reused, extended, or orthogonal-with-reason. Silence is not an answer (§11.4.118). 9. Each of the 7 Bridge layers is classified WIRED vs DESIGNED-ONLY with a captured probe. A design document existing is NOT evidence a layer is installed — that is precisely the §11.4.108 SOURCE-vs-RUNTIME gap, and reporting a documented layer as an available one would be a §11.4 bluff. 10. The skill-namespace collision risk is addressed with a designed mitigation: HelixSkills is itself a skills system, while §11.4.164's post_update_hook.sh already registers constitution skills into .claude/skills/. Two systems registering into one namespace needs defined precedence and clash handling, not discovery at runtime. 11. SuperSpec and SuperB are THIRD-PARTY, non-own-org dependencies

(§11.4.74 vendor path) — their health, maintenance status and supply-chain exposure must be assessed, not assumed from the design doc referencing them.

HXC-164 — Infrastructure startup script drives containers directly instead of through the shared containers component

Status: Queued **Type:** Task **Created-By:** Claude

The script that brings up the supporting services for testing talks to the container tooling directly, rather than going through the shared containers component the organisation maintains for exactly this purpose. That component exists so every project starts services the same way, gets the same health checking, and benefits from fixes once rather than repeatedly. Bypassing it means this project carries its own copy of that logic, which drifts and has already caused one outage of its own. A related leftover is that several optional service startups still hide their errors, so a failure to start looks identical to a success. This was flagged honestly in a commit message, but a note in a commit message is not an obligation anyone will act on, which is why it is being tracked here. Closing this means routing the startup through the shared component and removing the remaining error-hiding so a failed start is visible.

HXC-166 — The agent component has 204 known security advisories against its dependencies, including 5 critical

Status: Queued **Type:** Bug **Created-By:** Claude

When publishing the agent component, the hosting provider reported 204 outstanding security advisories affecting the libraries it depends on: 5 rated critical, 80 high, 100 moderate and 19 low. These are known, publicly documented weaknesses in third-party code the product ships and runs, so anyone who knows about them knows about them for our deployment too. Nothing here was introduced by recent work; the count has simply been accumulating and was surfaced by a routine publish. The risk is real but unquantified for us specifically, because a published advisory only matters if the affected code path is actually reachable in how we use the library. The right

next step is to pull the full advisory list, separate the ones we genuinely reach from the ones we do not, and upgrade or replace the dependencies behind the critical and high findings first. Until that triage is done we cannot honestly claim the component is free of known vulnerabilities.

HXC-168 — A database password is written directly into published setup files and has never been changed

Status: Queued **Type:** Bug **Severity:** High **Created-By:** Claude

The password used to reach the project database is written in plain text inside several setup and container files, and those files are published on all four public code-hosting accounts. Anyone who reads the project can read the password. This is not new and was not introduced by recent work; the project's own earlier security review already recorded it as something that must be replaced, and that has still not happened. The practical risk depends on whether the same password is used anywhere reachable from outside, which has not been established either way and should not be assumed harmless. Because the value is already public, changing the files alone is not enough — the password itself has to be changed everywhere it is used, and the setup files must then read it from a private configuration source rather than containing it. Until both halves are done the project cannot honestly claim its credentials are protected.

HXC-171 — A copied-in third-party web app carries most of our reported security warnings but cannot even be built

Status: Queued **Type:** Bug **Severity:** Medium

A complete copy of somebody else's web application sits inside the agent's source tree, and it alone accounts for one hundred and thirty-nine of the two hundred and four reported dependency security warnings, including three of the five most severe. It cannot currently be built or run: the startup configuration points at a build recipe file that does not exist in that folder, and its supporting packages have never been downloaded here. This matters in two opposite ways at once. Because it is not built, those warnings do not describe anything

we actually run today, which is why they rank low. But because a startup entry still references it, the setup is broken and misleading, and if anyone ever supplies the missing recipe file, all one hundred and thirty-nine warnings become live at once with no warning to whoever does it. Anyone reading our security reports benefits from removing this distortion, since it currently buries the small number of warnings that genuinely matter. The expected outcome is an explicit decision, recorded in writing: either keep the copy and properly maintain and build it, or remove it and its broken startup entry, or replace it with a reference to the upstream hosted service. Because deleting shipped components requires approval, the decision must be put to the operator rather than taken unilaterally.

HXC-172 — Three security warnings in a small plugin service we wrote and actually deploy, with no reachability evidence either way

Status: Operator-blocked **Type:** Task **Severity:** High

A small companion service we wrote ourselves is packaged into a container and started as part of the standard deployment, and it carries three known security warnings in its supporting packages, two of them rated high. Unlike the great bulk of the reported warnings, these have no mitigating evidence in either direction: we cannot say the code is unshipped, because it demonstrably ships, and no analysis has been run to establish whether the affected functionality is ever exercised. This matters because it is the one place in the whole review where something we genuinely run carries unassessed risk, which makes it far more deserving of attention than its small count suggests. One of the three is not really an upgrade problem at all: it concerns a protective setting that is switched off unless explicitly enabled, so updating the package without also turning the protection on would leave the exposure in place while appearing to resolve it. Users of any deployment running this service benefit. The expected outcome is a reachability assessment of the three warnings, the two straightforward package updates applied, the protective setting explicitly verified as enabled in our configuration rather than assumed, and the service rebuilt and confirmed working.

HXC-175 — An unreachable safety fallback in the model cache could silently serve stale data forever if edited

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

The cache that remembers model information has a small safety fallback for the case where its clock was never configured. That fallback can never actually run today, because the only way to build the cache always configures the clock — so it is untested and unexercised. On its own that is harmless. The reason it is worth recording is what happens if someone later edits it: a plausible-looking change to return an empty time instead of the current time would make every cached entry appear to have been fetched at a point so far in the past that the freshness check reads as a negative age, which never exceeds the expiry limit. Every entry would then be considered fresh forever, and the product would keep serving outdated model information indefinitely with no error and no failing test. The choice is to either add a small test that pins the fallback's behaviour, or remove the branch entirely and document that there is one supported way to construct the cache.

HXC-178 — The API key check compares secrets in a way that can leak them one character at a time

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

When a caller presents an API key, the server compares it to the configured key using an ordinary text comparison. That kind of comparison stops as soon as it finds a character that does not match, so a key sharing the first few characters takes measurably longer to reject than one that differs immediately. An attacker who can measure those tiny differences across many attempts can recover the key one character at a time without ever guessing it outright. Network timing noise makes this difficult in practice, which is why it is low rather than urgent, but it is a genuine weakness and the remedy is standard and cheap: use the comparison function designed for secrets, which always takes the same amount of time regardless of where the difference falls. This is pre-existing and was not introduced by recent work.

HXC-179 — A browser-origin check safely allows requests with no origin, but only because a separate login check is wired in

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude

The check that decides which websites may open a live connection to our server deliberately allows requests that carry no origin information at all. That is correct behaviour, because only browsers send that information and non-browser clients legitimately omit it. It is safe today only because a separate login check runs first and turns away anyone without valid credentials. The two protections are therefore coupled, but nothing records or enforces that coupling: if someone later removed or reordered the login check while working on something unrelated, every client that omits origin information would become completely ungated, and no existing test would notice. The work here is to add a check that specifically confirms the login step is still wired ahead of the connection upgrade, so the dependency is protected rather than merely true by luck.

HXC-180 — Three historical commits cannot be built, and this must be documented because it can never be repaired

Status: Queued **Type:** Task **Severity:** Low **Created-By:** Claude

Three commits in the current release range do not build. All three share one cause: a test referred to a piece of data that was not added until a later commit, so anyone checking out those three points gets a build error. The tip of the branch is fine and the problem was corrected shortly afterwards. This cannot be repaired, because repairing it would mean rewriting published history, which the project forbids outright. The practical consequence is for anyone hunting a regression by stepping backwards through history: they will hit three points that fail to build for a reason unrelated to whatever they are hunting, and may wrongly conclude the fault lies there. The work here is simply to record the three affected points and their shared cause somewhere a person doing that search will find it, so the failure is recognised immediately as known and irrelevant rather than investigated from scratch.

HXC-181 — Amend the constitution canon's superseded summary-generator naming and re-cascade to the 129 owned-submodule copies

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude **Assigned-To:** Claude

HXC-160 corrected the 21 stale references to scripts/generate_issues_summary.sh and scripts/generate_fixed_summary.sh across the 12 parent-repo governance carriers, and added the CM-SUPERSEDED-GENERATOR-NAMING gate that guards them. Two reference sets were deliberately left untouched and are the subject of this item. First, constitution/Constitution.md still carries 14 references across several different anchors, including universal rules that projects other than HelixCode consume and historical migration-plan records such as the Phase 5 Go-binary-shim plan. Amending canonical constitutional text is a governance amendment that CONST-049 requires be committed and pushed to every configured upstream, then re-cascaded and pointer-bumped. Pushing was forbidden in the session that closed HXC-160, and a canon amended locally but never propagated is worse drift than the stale wording, so the canon was left alone. Second, 129 files across roughly 45 owned submodules carry cascaded restatements of the same two anchor sentences. The correction applied in the parent repo is deliberately scoped with the words in HelixCode, because the fix names a HelixCode-specific mechanism. Copying that HelixCode-scoped wording into decoupled, project-agnostic submodules would inject project context into them and violate CONST-051(B). The correct sequence is therefore to first agree project-neutral canonical wording that names the workable-items DB exporter generically, land and push it in the constitution submodule per CONST-049, and only then re-cascade to the fleet and bump pointers. Who benefits: every consuming project whose manuals inherit the anchor text, and any agent reading a submodule manual to learn how tracker summaries are produced. Acceptance: canon names the DB exporter in project-neutral wording, the change is pushed to every constitution upstream, all 129 submodule copies are re-cascaded, submodule pointers are bumped, and the cascade verifier plus CM-SUPERSEDED-GENERATOR-NAMING both pass with scope widened to the fleet.

HXC-186 — Evidence files can accidentally satisfy the evidence requirement for an unrelated change

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude

Every shipped change must carry recorded proof that it works, and a check enforces that by looking for the change's identifier inside the evidence files. The evidence files also automatically stamp in whatever the project's current position was at the moment they were recorded. That stamp is an identifier of exactly the kind the check looks for, so an evidence file written while the project happened to be sitting at some change can, purely by coincidence, be accepted as that change's proof even though it demonstrates something completely unrelated. This has not yet happened, because every recording so far was made while the project sat at a position that requires no evidence, but the coincidence is available to every recording the shared tooling produces. The fix is to make the check accept only identifiers deliberately declared as what the evidence is for, ignoring the incidental position stamp. This needs care because tightening the rule may withdraw acceptance from evidence already recorded, so it is a deliberate decision rather than a quiet correction.

HXC-188 — The project handbook describes a folder layout the project no longer has

Status: Queued **Type:** Task **Severity:** Low **Created-By:** Claude

The main handbook that tells contributors and automated helpers how this project is organised lists several folders at the top level that do not exist. It names four internal areas when only one is present, and refers to a top-level commands folder that is absent entirely. Anyone following it — a new contributor, or an automated helper reading it for orientation — will look for things that are not there, and may conclude the project is broken or that they have checked out the wrong thing. It also undermines trust in the rest of the document, which is otherwise the authoritative description of how to work here. The correction is small and purely descriptive: measure the actual layout and update the section to match, then keep the two in step as the layout changes.

HXC-189 — A dependency is pinned to an old project name that only works because of a redirect

Status: Queued **Type:** Task **Severity:** Low **Created-By:** Claude

One of the external projects this codebase depends on has been renamed. Our configuration still refers to it by its previous name, which currently works because the hosting service silently forwards requests from the old name to the new one. That forwarding is a courtesy, not a guarantee: it disappears if anyone ever creates a new project under the old name, and it is not honoured by every tool that might fetch the dependency. If it stops working, fetching the project fails with an error that points at a name nobody recognises, which is unusually confusing to diagnose. The fix is to record the project's current canonical name so the reference stands on its own rather than depending on a redirect continuing to exist.

HXC-190 — Commands stopped by a timeout are reported as though they were not

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

When a running command is cut short because it took too long, the result that comes back says it was not a timeout. The check that sets that flag looks at the wrong clock: it inspects the caller's own deadline, which usually has none at all, rather than the timer that actually stopped the command. Investigation showed the obvious alternative would not work either, because the internal signal it would consult can only ever report a plain cancellation and never a timeout, and the timer that does the stopping is a separate mechanism entirely. The effect is that anyone diagnosing a slow command sees a generic failure and no indication that a time limit was the cause, which sends them looking in the wrong place. A sibling code path that runs commands the ordinary way sets the flag correctly by doing it inside the timer's own callback, so a working pattern already exists to copy.

HXC-191 — Cancelling a command cannot actually stop it when the protective sandbox is switched off

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

When a command is cancelled or times out, the system tries to stop not just that command but anything it started, by signalling the whole group of processes together. That only works if the command was placed into its own group when it began. With the protective sandbox enabled — the normal configuration — it is, and cancellation works. With the sandbox switched off, the grouping step is skipped but the stop attempt still asks the operating system to signal a group that does not exist, which fails silently because the error is discarded. The result is a cancelled command that keeps running. Today this is dormant, because the only configuration that disables the sandbox is used by a single test, so no shipped path reaches it. It is worth fixing because the failure is completely silent and would appear only in an unusual configuration, which is the hardest kind of problem to diagnose later.

HXC-192 — A single over-long line of output permanently stops reading, which can freeze the command producing it

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

Output from a running command is read line by line, with a limit on how long any single line may be. When a line exceeds that limit the reader stops — not just for that line, but permanently for that stream, because the buffer is fixed at the limit and can never grow past it. Nothing else takes over the reading. The command on the other end keeps writing until the operating system's own small buffer fills, and then it blocks forever waiting for someone to read. So one unusually long line, which programs emitting compact machine-readable output produce quite naturally, can wedge the command that produced it. The fix is to keep draining and discarding the remainder of an over-long line rather than abandoning the stream, so the producing command is never starved of a reader.

HXC-196 — The project handbook names a component version the code no longer uses

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude

The handbook that describes this project's technology stack names a specific version of one of its networking libraries. Both parts of the codebase now use a newer version, because a security fix required moving off the older one. The upgrade was correct and deliberate; it is the handbook that is now wrong. This matters because that section is what a new contributor or an automated helper reads to learn what the project runs on, so it will confidently state a version that has not been used for some time — and because the same document has already been found describing folders that no longer exist, each additional inaccuracy makes the whole document less trustworthy. The fix is to update the recorded version to what is actually in use, and to note that the security upgrade is the reason, so nobody later reverts it believing the handbook was correct.

HXC-197 — A directory tree contains a stale copy of itself nested several levels deep

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

One test directory contains a duplicate of its own path nested inside itself, so the same ten files appear twice in the project at two different depths. It was created by an earlier renaming exercise that moved a tree while leaving a copy behind at the old location inside the new one. Nothing reads the nested copy, so it causes no failure today, but it means anyone searching the project finds two results for every one of those files and cannot tell which is real, and any tool that scans for duplicate declarations reports it forever. It also blocks a useful check from being switched on: a guard that detects accidentally-nested duplicates cannot be enabled while this one exists, because it would report a failure on every run. Removing it requires first confirming through the project history that the nested copy is genuinely the leftover and not the original.

HXC-199 — A check for a just-fixed problem would still report it as broken, because it matches on a fragment

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude

A recent change gave one part of the project a distinct name so that two different pieces of code could stop claiming the same identity. A diagnostic script that checks whether that problem still exists searches for the old name as a fragment of text, and the new name contains the old one as a prefix — so the check still matches and would report the problem as unfixed even though it is fixed. Anyone re-running it would be told to redo work that is already done, and might undo the fix believing it had never worked. The project handbook also still describes the old arrangement in three places. The fix is to make the check match the whole name rather than a fragment, and to update the handbook so both agree with what the code actually does now.

HXC-200 — A leak detector recognises only one of the two shapes the leak can take

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

A detector was written to catch a specific kind of stall where a background worker gets stuck trying to hand off a result nobody is collecting. It identifies that stall by looking for one particular description of what the worker is waiting on. There is a second, equally real form of the same stall — where the worker waits on a choice between several possibilities rather than a single handoff — and the detector does not recognise it. This was demonstrated rather than theorised: a deliberately introduced leak of the second form went completely undetected while the detector reported everything healthy. The shape it does catch is the historical one, so the guard is not useless, but a future change that keeps the newer structure while breaking its escape route would slip past. Widening it to recognise both descriptions is a small change and restores the guard to covering the whole defect rather than half of it.

HXC-202 — The network address fix reached the server but not the app that connects to it

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude

A recent change taught the product to correctly format modern internet addresses when combining them with a port number. It was applied where the server decides what address to listen on, but not in the desktop application for one platform that builds the address it connects to from the very same two configuration settings. So with such an address configured, the server starts up correctly while that application constructs an unusable address and cannot reach it — a failure that looks like the server is down when it is running perfectly. A second instance exists in a maintenance tool that contacts a code-analysis service. Neither is newly broken; both were simply outside the list of places the original change examined, so nothing was made worse. This is the seventh time in one working cycle that a correct change was applied in one place and not to a sibling doing the same thing nearby, which is worth treating as a review habit rather than seven separate oversights.

HXC-204 — Two more endurance tests report failure whenever the machine is busy

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude

Two long-running tests — one exercising heavy simultaneous access to the memory store, one exercising leader election among worker nodes under deliberately induced faults — report failure when the machine is under load, and pass comfortably when run on their own. Measured: during a full test run both failed, while in isolation they passed three times consecutively, one finishing in roughly a fifth of its allowed time and the other in about a twentieth. Their verdicts are therefore reporting how busy the machine was, not whether the product works. This matters most at exactly the wrong moment: the complete test run performed before a release is by definition the busiest the machine ever gets, so these two will mark a healthy release as broken and train everyone to wave the result through. A third test with the same problem was already corrected by making an inconclusive run report as skipped rather than failed, while keeping its real checks strict; the same treatment fits here. Both should wait for the condition they care about rather than assuming a fixed time is enough.

HXC-206 — A provider stopped and restarted during a health probe can have a stale verdict applied

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

The newly-fixed health refresh deliberately releases its lock while it contacts each provider over the network, so that a slow or hanging provider cannot block everyone else. It then re-acquires the lock and only applies its verdict if the provider's state has not changed underneath it. There is a narrow remaining window: if a provider is stopped and started again entirely within one probe, the check sees the same state it started with and applies a verdict formed against the previous instance. The result would be a provider briefly reported unhealthy when it is fine, which the next refresh corrects, and the dangerous direction — reporting a stopped provider as healthy — is already prevented. Worth recording that this is cheap to close rather than expensive: a hidden counter incremented on each state change is invisible to the outside world and requires no change to any published structure. An earlier note describing it as a published-interface change was wrong and has been retracted.

HXC-207 — Every model manager shares one settings map by reference, so a future write would corrupt all of them

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

When a model provider record is created, its environment settings are taken from a shared template by reference rather than copied. Every record in every manager therefore points at the same underlying settings, including across separate manager instances. Nothing writes to those settings today, and the values handed out to callers are copied, so no corruption occurs at present — this was verified rather than assumed. The reason to record it is that the protection now added to these managers is per-instance: a lock in one manager cannot protect a structure shared with another. So the day someone adds a legitimate, properly-locked write to a provider's settings, it will silently alter every other manager's view as well, and the locking will look correct while failing. Copying the settings when the record is created removes the hazard permanently and costs nothing.

HXC-208 — Two mock services are wired into e2e compose stacks that cannot build them

Status: Queued **Type:** Bug **Created-By:** Claude

Four compose entries across two end-to-end test stacks declare a build context pointing at the Slack mock and the LLM-provider mock, but neither directory contains a Dockerfile. Any attempt to build those stacks therefore fails at the build step rather than at run time, which means the containerised form of these two services has almost certainly never been exercised. This was surfaced while fixing an unrelated permissive-origin defect in the same two services: the fix could be proven at the source and test-process level but not against a running container, precisely because no container of either service can be produced. That gap matters beyond these two mocks, because a fix validated only in-process leaves the deployed-artifact layer unproven, which is exactly the class of silent failure the four-layer verification discipline exists to catch. The work is to add the missing Dockerfiles, or to remove the build entries if these services are genuinely never meant to run containerised, and then to prove the choice by building the stack.

HXC-211 — Four more unbounded waits remain in the shell package after the latest hang fix

Status: Queued **Type:** Bug **Created-By:** Claude

The hang just repaired was one instance of a pattern that survives in four other places in the same package, each able to park a call forever under conditions the code already permits. Two sit in the synchronous execution path and become unbounded when sandboxing is switched off or when no timeout is supplied, both of which are reachable through documented usage rather than misconfiguration. Two more sit in the output-streaming helper and are presently safe only because their single existing caller happens to rescue them, so any future caller wiring that helper the obvious way reproduces the original defect exactly. A fifth site in the background task manager amplifies all of them by calling into this code with no timeout and no way to abandon the attempt. These were found by sweeping the whole package after

the fix rather than only the function named in the report, which was necessary because the reported defect and its already-fixed twin turned out to live in different files, so a reviewer re-reading the same file would have found nothing.

HXC-213 — The deployment tracker guards one line of shared state and leaves seventy-seven unguarded

Status: Queued **Type:** Bug **Created-By:** Claude

A component that tracks the progress of a production deployment declares a lock for protecting its shared status record, and then uses that lock in exactly one place while touching the same record in seventy-eight others. This is the same shape as a defect just fixed elsewhere, and it is not speculative: a comment in the file records that a real race was previously detected here by the race detector, and the repair made at that time guarded only the single line the detector happened to point at, leaving every other access untouched. So the file now reads as though it is synchronised, which is more misleading than having no lock at all, because a reader sees the mechanism and assumes it applies. Concurrent deployment phases can therefore overwrite each other's status, and a deployment can report a state that never occurred. The remedy is the one already proven twice in this codebase: bring every access to the shared record under the existing lock, keeping the lock away from any long-running call, and add a guard that runs under the race detector.

HXC-214 — Four model providers hand callers a live pointer to their own health record

Status: Queued **Type:** Bug **Created-By:** Claude

Four provider integrations each declare a lock and then, in a method whose name promises only to read, both modify their stored health record and hand the caller a pointer to that same record rather than a copy. Two faults follow from one line. A method that reads according to its name but writes in practice will be called freely from places that assume it is safe, so the write happens under no lock and concurrent callers overwrite each other. And because the caller receives the live

pointer rather than a copy, anything it does to the value it was given silently changes the provider's own state, which no caller could reasonably expect. The result is that a provider can be reported healthy when it is failing, or the reverse, and that an unrelated piece of code can corrupt that judgement without ever intending to touch it. The remedy matches the two fixes already landed for this pattern: perform the update under the lock and return a copy, then guard both halves with a test that fails if the returned value is ever aliased to the stored one.

HXC-215 — The commit-compile gate blames a different innocent commit on each run

Status: Queued **Type:** Bug **Created-By:** Claude

The gate that checks whether each recent commit can actually be built reported a failure on two separate runs, naming a different commit each time, and in both cases the named commit builds cleanly when the gate's own command is re-run against it in isolation. The packages it reported as broken were unrelated to anything the accused commits changed, which was the first clue. The gate has no time limit, so these were real non-zero results at the moment they happened rather than a run that was cut short. What distinguishes the gate's own execution from a reproduction is that it builds five commits back to back, each in a fresh throwaway checkout, all of them sharing one build cache, whereas a reproduction builds one. That points at interference between those consecutive builds rather than at any defect in the code being judged, though the precise mechanism is not yet proven and should not be assumed. This matters more than an occasional wrong answer suggests, because the gate blocks releases: a check that accuses innocent work teaches people to dismiss it, and the day it is right about a genuinely broken commit that habit will let the breakage through. The work is to make the gate produce the same verdict every time it is given the same input, and to prove that by running it repeatedly against an unchanged tree.